

Project 1 – Neural Networks

Due: 6/24/2026

Important Note: This project will require a significant amount of self-learning. You will need to develop a solid understanding of PyTorch, including its core components such as tensors, autograd, and optimizers, as well as fundamental deep learning concepts such as epochs and batch gradient descent. You can (and probably should) use AI tools to help write or understand PyTorch code and these concepts. But it is important that you fully understand the material yourself and are able to explain it.

1. Gradient descent on neural networks

Consider the simple neural network discussed in Notebook 06: two inputs x_1, x_2 , a single hidden layer of four neurons, and one output neuron. Recall that each hidden neuron forms a pre-activation and an activation,

$$z_j = w_{1j} x_1 + w_{2j} x_2 + b_j, \quad a_j = \psi(z_j),$$

and the output neuron produces

$$h = c + u_1 a_1 + u_2 a_2 + u_3 a_3 + u_4 a_4, \quad y = \psi(h).$$

Assume that the loss is $L = \frac{1}{2}(d - y)^2$, where the target d is a constant, and the activation function ψ is the sigmoid. Using the chain rule, derive the following partial derivatives of the loss, and then state the corresponding gradient-descent update rule (using learning rate η) for each weight:

(a) $\frac{\partial L}{\partial u_1}$, (b) $\frac{\partial L}{\partial w_{24}}$.

Derive each ingredient one by one as we did in Notebook 05.

2. The digit classifier

In Notebook 06 we have discussed and seen an example of a neural network: neurons form a weighted sum, an activation function applies nonlinearity to the result, and layers are stacked together. In this project, you will build a real neural network in PyTorch and train it to recognize handwritten digits. The notebook explains each task in detail and provides the data visualizations and the result plots. You only write the marked code cells.

The model you must build. Implement a fully-connected feed-forward network with the architecture below. Each image is flattened from its 28×28 grid into a vector of 784 values before entering the network. Size simply means the number of neurons in that layer. You'll find more details in the notebook.

Layer	Size	Activation
Input	784	–
Hidden layer 1	32	ReLU
Hidden layer 2	16	ReLU
Output	10	None (raw class scores)

3. Deliverables

- Your derivations for Part 1 (both partial derivatives and their update rules).
- Your completed `project1.ipynb`, runnable top to bottom so every cell shows its output.